

Natural Language-Based Command Option Retrieval Using a Local-First Approach: A Proof of Concept

Leon Eltrich & Luaine Le Cadet
DHBW Ravensburg

Introduction & Contribution

Command-line interfaces (CLIs) are powerful but notoriously non-intuitive due to cryptic, inconsistent syntax. While cloud-based AI assistants can translate natural language into commands, they introduce severe privacy risks, fail in offline environments, and lack knowledge of proprietary, internal tools.

To bridge this gap, we introduce a local-first, offline CLI assistant. Operating entirely without background daemons or cloud dependencies, it securely translates natural language into syntactically valid commands in under a second. Built on a modular Hexagonal architecture, the system allows developers to securely ingest custom tool catalogs while guaranteeing that commands are strictly recommended as text and never autonomously executed.

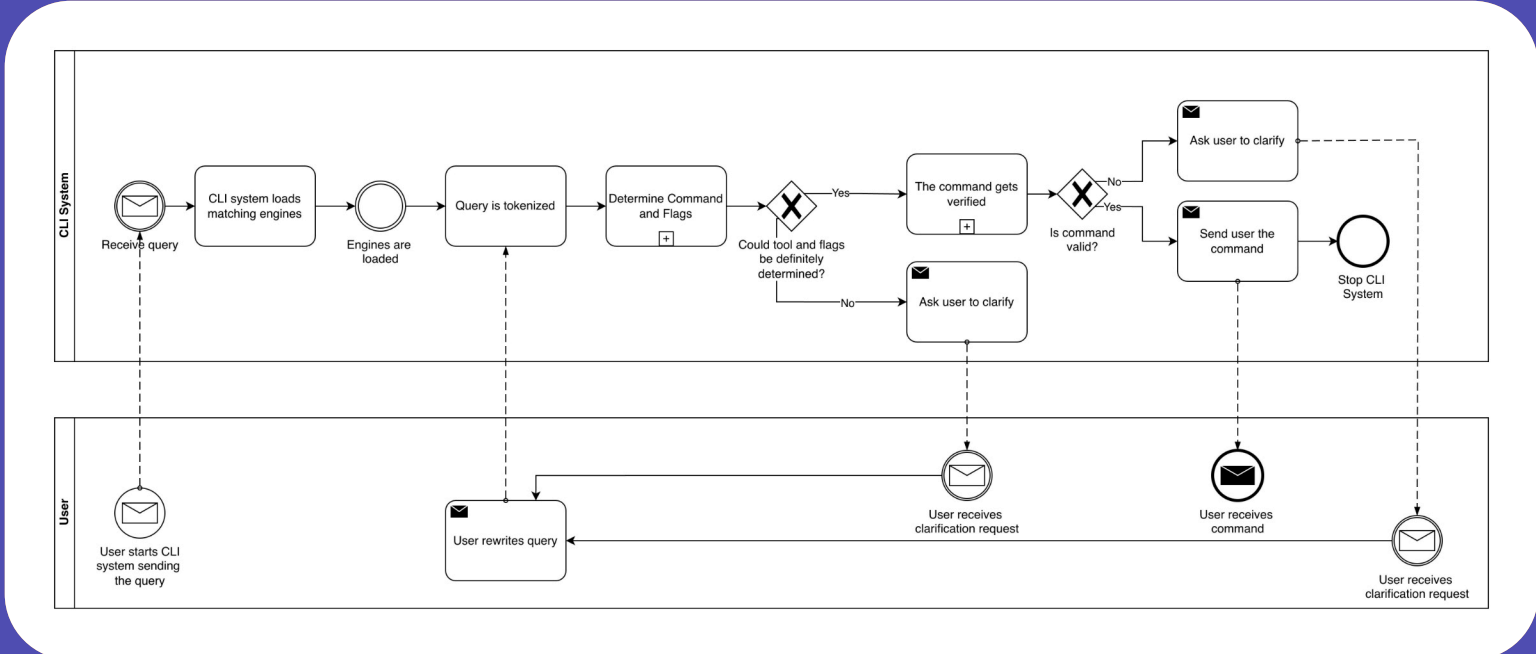
Privacy & Security

Knowledge Gaps

Offline Constraints

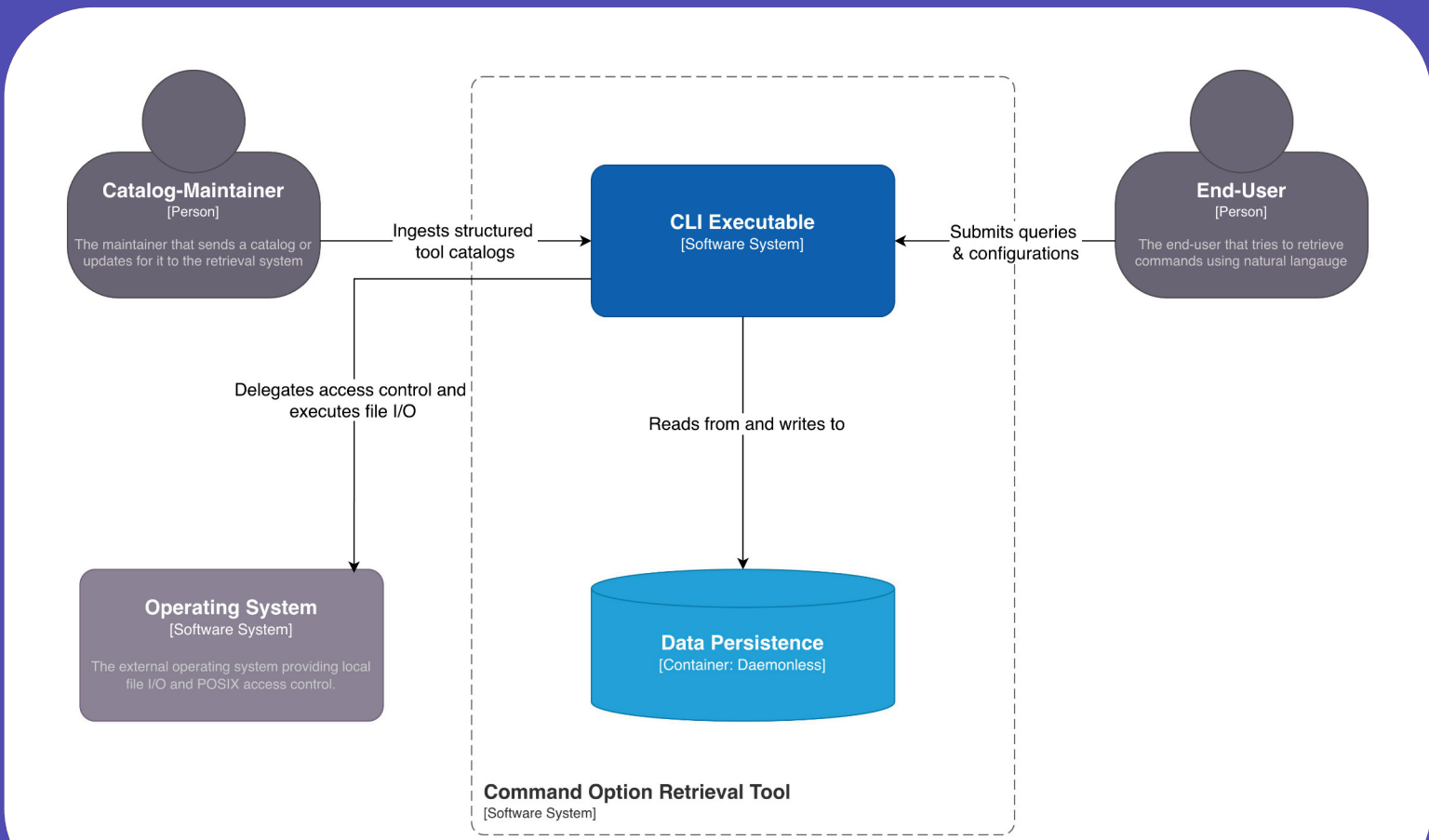
Dynamic Execution: The Query Resolution Flow

The execution flow begins the moment a user submits a query, prompting the system to immediately load its matching engines and tokenize the raw input. The system then evaluates these tokens to determine the appropriate command and option flags. If it cannot definitively determine the target tool, it halts and asks the user for clarification. When the user rewrites their query, the system efficiently bypasses the initial engine-loading phase and routes the new input directly back to tokenization. Once a command is successfully identified, it undergoes a final syntactical verification to ensure it is valid. If this validation fails, another clarification loop is triggered. Conversely, if the command is structurally sound, the finalized text is sent back to the user, and the system immediately terminates its execution to conserve host resources.



Architectural Boundaries & Embedded Persistence

To guarantee absolute privacy, the system establishes a strict offline perimeter within the host Linux environment. It executes zero outbound network requests, ensuring user queries and proprietary catalogs remain entirely local. External interactions are restricted to the End-User via standard I/O and the Catalog-Maintainer via an authenticated ingestion interface. To enforce a zero-dependency philosophy and a strict memory ceiling, the architecture avoids standalone background servers. Instead, a volatile CLI executable pairs with a daemonless, embedded data store, initializing Just-In-Time and terminating immediately after query resolution.



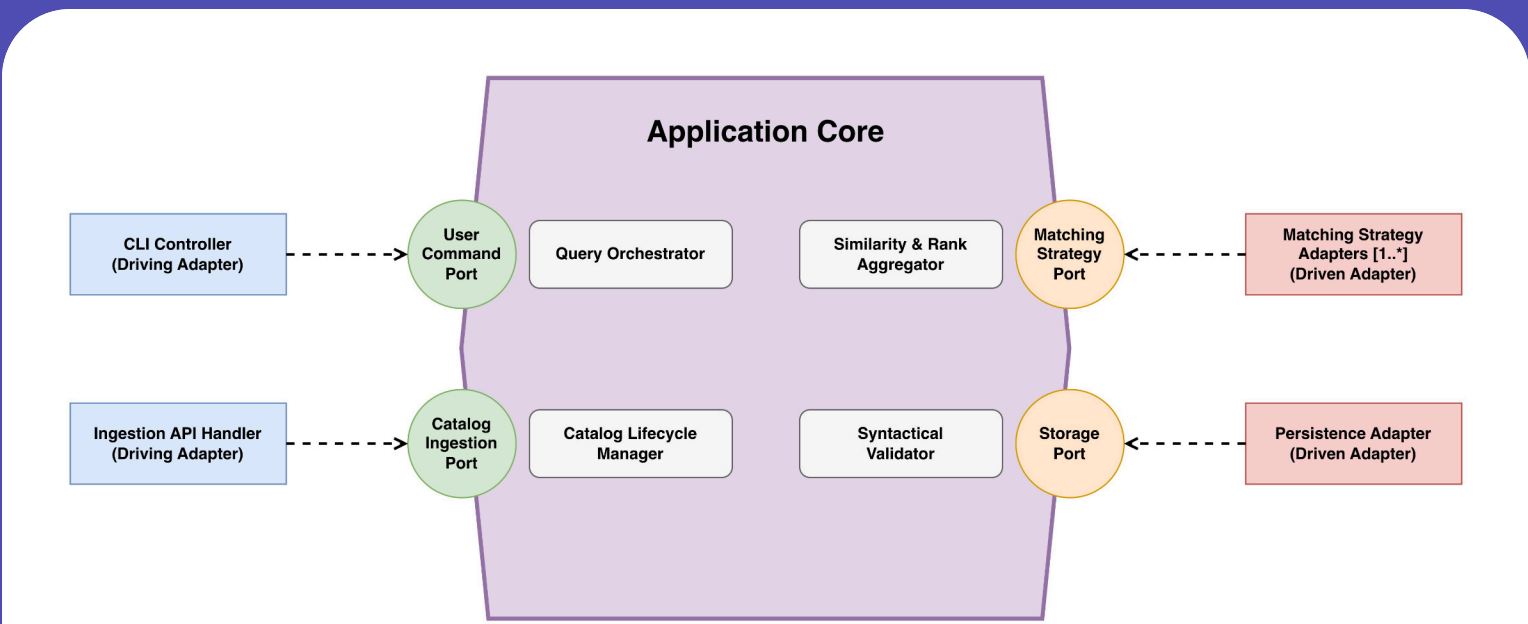
Architectural Design: The Hexagonal Model

To achieve strict performance budgets (< 1000ms latency, < 1024MB RAM) while maintaining high modularity, traditional N-Tier (layered) designs were discarded. Standard layered architectures inherently couple core business logic to the underlying database, while enterprise alternatives (like Clean Architecture) introduce unjustifiable computational overhead and boilerplate.

Instead, the system utilizes a Hexagonal (Ports and Adapters) Architecture, relying on the Dependency Inversion Principle to divide the application into two strict conceptual zones:

The Core (Inside): Encapsulates the pure domain logic, syntactical command validation, and mathematical rank aggregation. It possesses absolute zero knowledge of the external world, defining its data and execution requirements purely through abstract interfaces (Ports).

The Infrastructure (Outside): Encompasses all external mechanisms, including the user-facing CLI terminal, the data store, and the specific text-matching engines. These mechanisms act as Adapters that implement the Core's interfaces.



Finding The Engines

The system employs a multi-strategy approach that fuses scores from disparate ranking engines for maximum accuracy. Because of our Hexagonal Architecture, these engines function as completely isolated plugins. This structural decoupling allows us to easily test, benchmark, and hot-swap different models without modifying any core logic.

Finding The Cutoff Point

Once the core fuses the plugin results into a single, unified ranking, the system must determine the optimal cutoff point to drop low-confidence candidates. We will evaluate and compare filtering approaches to establish a dynamic "cliff-severing" boundary. This ensures the system includes all relevant options while strictly filtering out low-quality hallucinations.